Day 11: Log Filtering and Pattern Detection

Identifying Repeated Usernames, Source IPs, and Failure Patterns with grep, awk, and sort

Field	Details
Intern Name	Amrita Pathak
Date	13 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	13 September 2026, 11:59 PM IST

Table of Contents

Table of Contents 2
Executive Summary 3
1. Objective 3
2. Data Source Declaration 3
3. Tools and Environment 4
4. Key Terms 4
5. Why This Task Matters 5
6. Command Reference 5
6.1 Creating and inspecting the file 5
6.2 Filtering and counting 5
6.3 Ranking the results 6
7. Step-by-Step Execution 6
8. Pattern Summary 12
9. Findings 12
9.1 One dominant IP correlates with the two most-targeted usernames 12
9.2 The second source was smaller in volume, not narrower in targeting 12
9.3 The successful account never appears in either failure ranking 12
9.4 awk's associative arrays performed the counting without needing a separate tool 12
10. Conclusion 13

Executive Summary

This task used grep, awk, and sort to turn a small authentication log into a frequency-based finding, identifying which usernames, source IPs, and failure messages repeated most often. A small synthetic SSH authentication log was created directly on the Kali lab VM using a heredoc, a standard bash technique for writing multi-line text into a file, deliberately built with repeated usernames and two distinct source IPs so that every pattern type the task lists could be demonstrated cleanly in one dataset. This is disclosed openly here and again in Section 2, since knowing exactly what kind of data a finding was built on is part of reading it honestly.

Filtering the sixteen-line log surfaced fourteen failed authentication attempts spanning two distinct source IPs and six distinct usernames, alongside two legitimate successful logins from a separate, internal address. One IP, 203.0.113.5, accounted for eleven of the fourteen failures, nearly four times more than the second IP, and was also the source behind the majority of attempts against the two most repeated usernames, root and admin. That correlation, one dominant source repeatedly trying a short list of high-value usernames, is the actual signature this task was built to teach an analyst to recognize.

Every count was produced using only the three tools the task specifies, grep to isolate relevant lines, awk to tally repeated field values using its own associative arrays, and sort to rank the results, without relying on additional utilities such as uniq that are not part of the stated toolset.

1. Objective

Create a small sample authentication log and use grep, awk, and sort to identify repeated usernames, repeated source IPs, and repeated failure messages within it, surfacing patterns that would be relevant to a SOC analyst reviewing real authentication activity.

2. Data Source Declaration

The log file analyzed in this report, sample_auth.log, is a synthetic file created specifically for this exercise. It is not captured network traffic and does not represent a real attack against any system. It was written by hand on the Kali lab VM in the same line format real sshd authentication logs use, with sixteen entries deliberately chosen to include repeated usernames, two distinct source IPs, and a mix of failed and successful logins, so that every pattern category this task asks for could be demonstrated clearly in one small, fully understood dataset.

This choice is explicitly permitted by the task, which allows creating a small sample log file as an alternative to using live captured data. Building and testing a filtering approach against a dataset where the correct answer is already known, before ever pointing the same commands at real, unknown data, mirrors how detection logic is actually validated in professional practice. Every figure and finding in this report should be read as a demonstration of the filtering technique, not as evidence of a real security incident.

3. Tools and Environment

Item	Detail
Endpoint used	Kali Linux VM (kali)
Log file	sample_auth.log, a synthetic SSH authentication log created for this exercise
Creation method	bash heredoc (cat > file << 'EOF' ... EOF)
Tools used	grep, awk, sort
Log size	16 lines, 14 failed and 2 accepted authentication attempts

4. Key Terms

- **Heredoc**: a bash technique that lets multiple lines of text be written into a file directly from the terminal, using a marker such as EOF to indicate where the text begins and ends. It is a standard, everyday shell feature, not a workaround, commonly used to generate configuration files, scripts, or test data without opening a separate text editor.

- **Field ($1, $2, ...)**: awk automatically splits every line it reads into fields based on whitespace, numbered from 1. A line's ninth word is $9, its eleventh word is $11, and so on. This only works reliably when every line shares the same structure, which is why the sample log was written with a consistent format throughout.

- **Associative array**: a variable in awk that stores values under named labels rather than numbered positions, similar to a small lookup table. Writing count[$9]++ increases the counter stored under whatever username appears in that line, building up a running total for every distinct value seen.

- **END block**: a section of an awk program that runs exactly once, after every line in the input has already been processed, rather than running again for each line. It is the correct place to print final totals, since the counts are not complete until the whole file has been read.

- **sort -rn**: sorts input, treating each line's leading value as a number (-n) rather than as text, and reverses the order (-r) so the largest number appears first. Without -n, sort would compare values as text and could rank 9 above 11 simply because the character 9 is textually larger than the character 1.

- **Brute force/dictionary attack (MITRE ATT&CK T1110)**: a technique where an attacker systematically attempts many username and password combinations against a login service, hoping one succeeds. A single IP repeatedly targeting several likely usernames, root and admin, especially, is the classic log signature of this technique, which is what this task's findings ultimately identify.

5. Why This Task Matters

Every task up to this point involved reading a small, known number of events individually. This task is different in a way that matters a great deal in real SOC work: it tests whether raw, high-volume log text can be turned into a countable answer, not just read line by line. The specific benefits this builds are worth stating directly:

- **Frequency analysis is the foundation beneath real detection rules**: before any SIEM dashboard or alert exists, someone has to know how to ask a log file, in plain text, which value appears too often. grep, awk, and sort are the raw mechanics underneath that question, and understanding them directly makes it possible to sanity-check what a detection tool is actually doing under the hood.

- **A single failure is noise; a repeated one is a signal**: one failed login proves nothing on its own, a real person mistypes passwords constantly. The same username or the same source IP repeating across many lines in a short window is the actual signature analysts are trained to look for, and this task practices producing that count directly rather than eyeballing a log and guessing.

- **Correlating two patterns is stronger evidence than either alone**: a repeated IP by itself could be a misconfigured script retrying automatically. A repeated username by itself could be one person struggling to remember a password. The same IP repeatedly targeting the same small set of usernames, seen together, is what turns two weak signals into one strong one.

- **Building and testing against known data is a real professional habit**: writing a small synthetic log with a known, intended answer before running a detection command against real production data is exactly how detection logic gets validated in practice, confirming the command finds what it is supposed to find before trusting it on data where the answer is not already known.

6. Command Reference

Before walking through each step, this section explains every command and syntax element used in this task on its own, so the step-by-step section that follows can stay focused on what was run and what it found, rather than re-explaining syntax at every step.

6.1 Creating and inspecting the file

- `cat > file << 'EOF'` ... `EOF`: opens a heredoc, writing every line typed afterward into the named file until a line containing only EOF is reached. Nothing is read from an existing file; this creates a brand new one from typed text.

- `cat file`: prints a file's entire contents to the screen. Used here purely to visually confirm the file was written correctly before running any analysis against it.

6.2 Filtering and counting

- `grep -c "pattern" file`: counts how many lines in a file contain the given text pattern, printing only the number, not the matching lines themselves.

- `grep "pattern" file`: prints the full text of every line containing the given pattern, filtering out everything else. Used here to narrow sixteen lines down to just the fourteen failures, or just the two successes, before counting anything further.

- `$9` and `$11`: refer to the ninth and eleventh whitespace-separated words on each line that awk reads. In this log's consistent format, the username always lands in position 9, and the source IP always lands in position 11.

- `count[$9]++`: increases, by one, the value stored in the associative array count under whatever username appears in the current line. Repeated over every line, this builds a running total per username without needing to sort or group anything first.

- `END {for (u in count) print count[u], u}`: runs once, after all lines have been read, looping over every distinct username the count array collected and printing its final total next to its name.

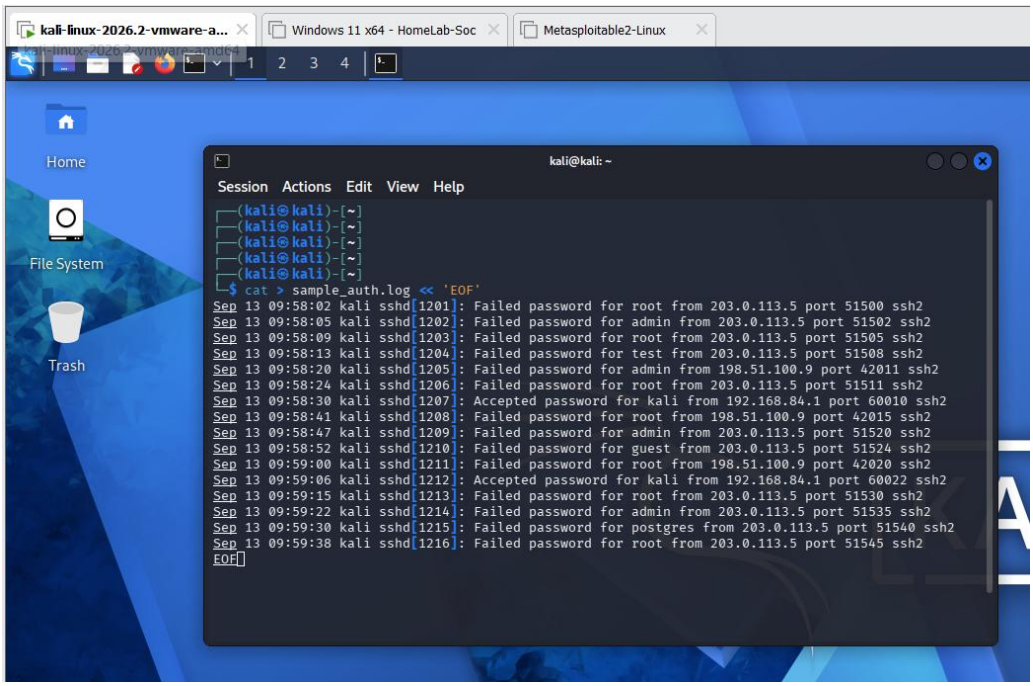
6.3 Ranking the results

- `sort -rn`: orders the piped-in lines by their leading number, largest first. -n ensures numbers are compared as numbers rather than as text, and -r reverses the default smallest-first order.

7. Step-by-Step Execution

Each step below states the command run and the direct result. The meaning of each command's syntax was already explained in Section 6.

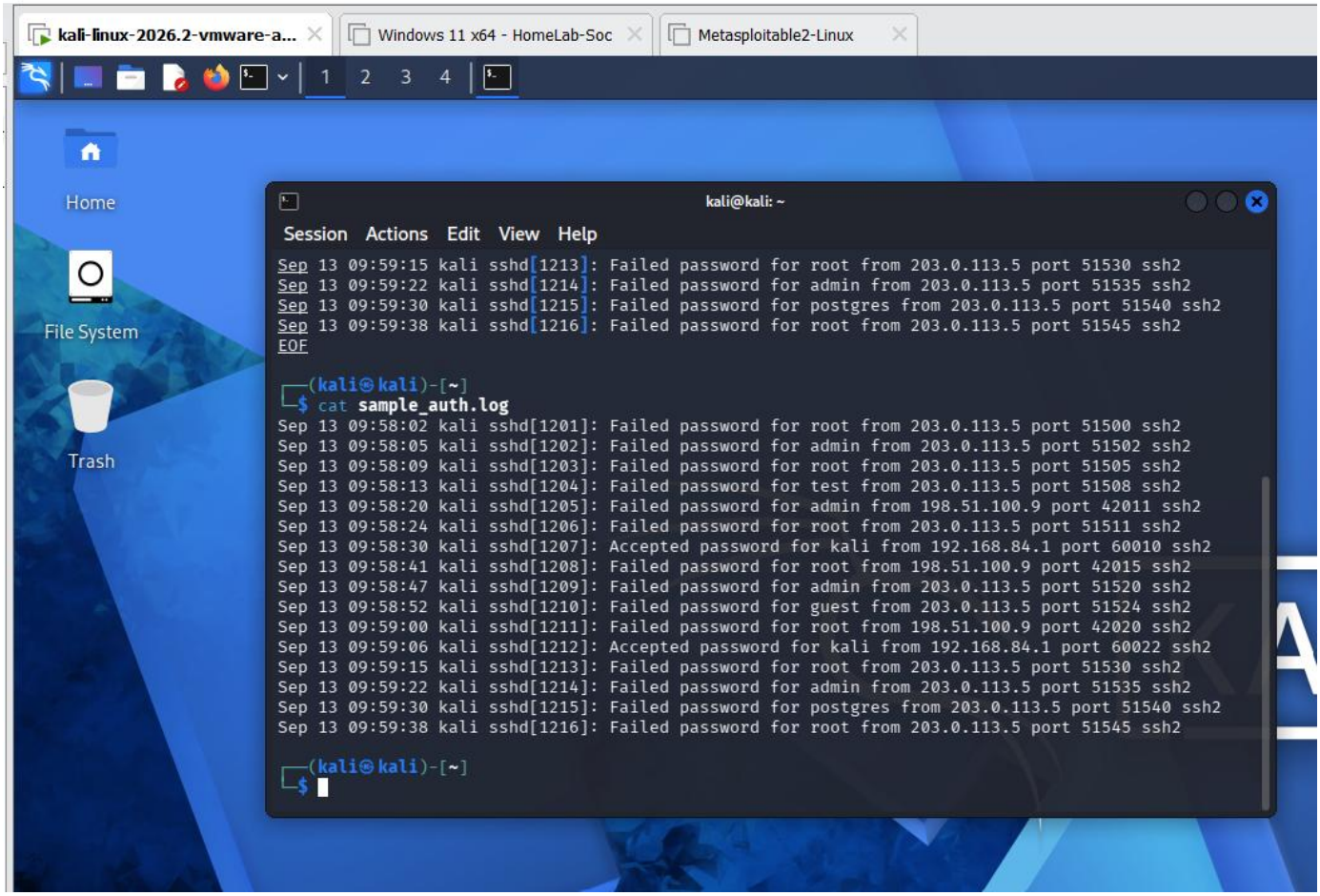
- **Step 1 - created the sample log with a heredoc:** ran `cat > sample_auth.log << 'EOF'`, typed sixteen synthetic SSH log lines, then closed with `EOF`. **Note: Synthetic data is created for this exercise, not captured network traffic. See Section 2, Data Source Declaration, for full disclosure.**



The heredoc in progress, sixteen synthetic SSH log lines being written into sample_auth.log, closed with the EOF marker.

- **Step 2 - confirmed the file saved correctly:** ran `cat sample_auth.log`.

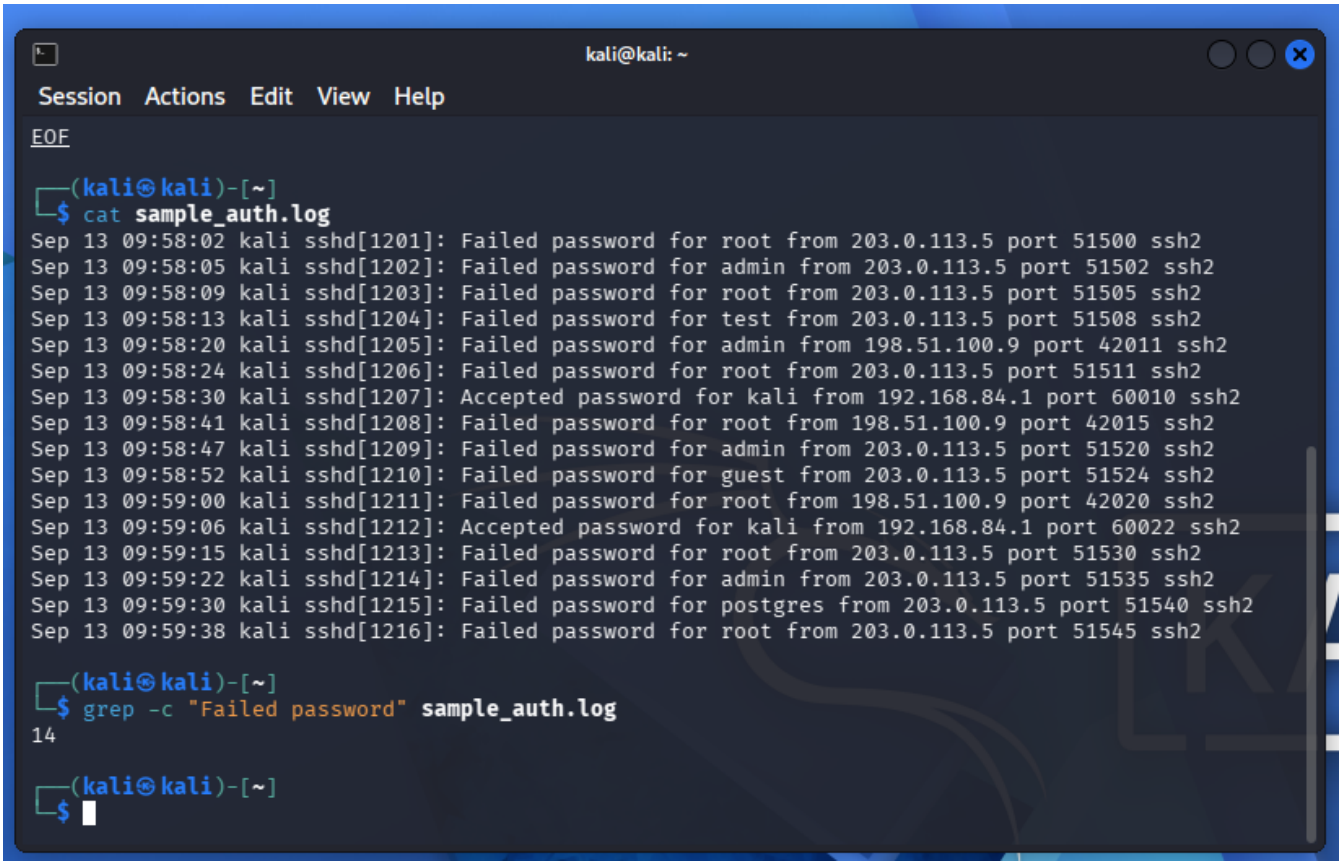
  Result: all sixteen lines printed back exactly as written, fourteen Failed password lines, and two Accepted password lines.



The full contents of sample_auth.log confirmed: sixteen lines, fourteen failed and two accepted authentication attempts.

- **Step 3 - counted the repeated failure message:** ran `grep -c "Failed password" sample_auth.log`.

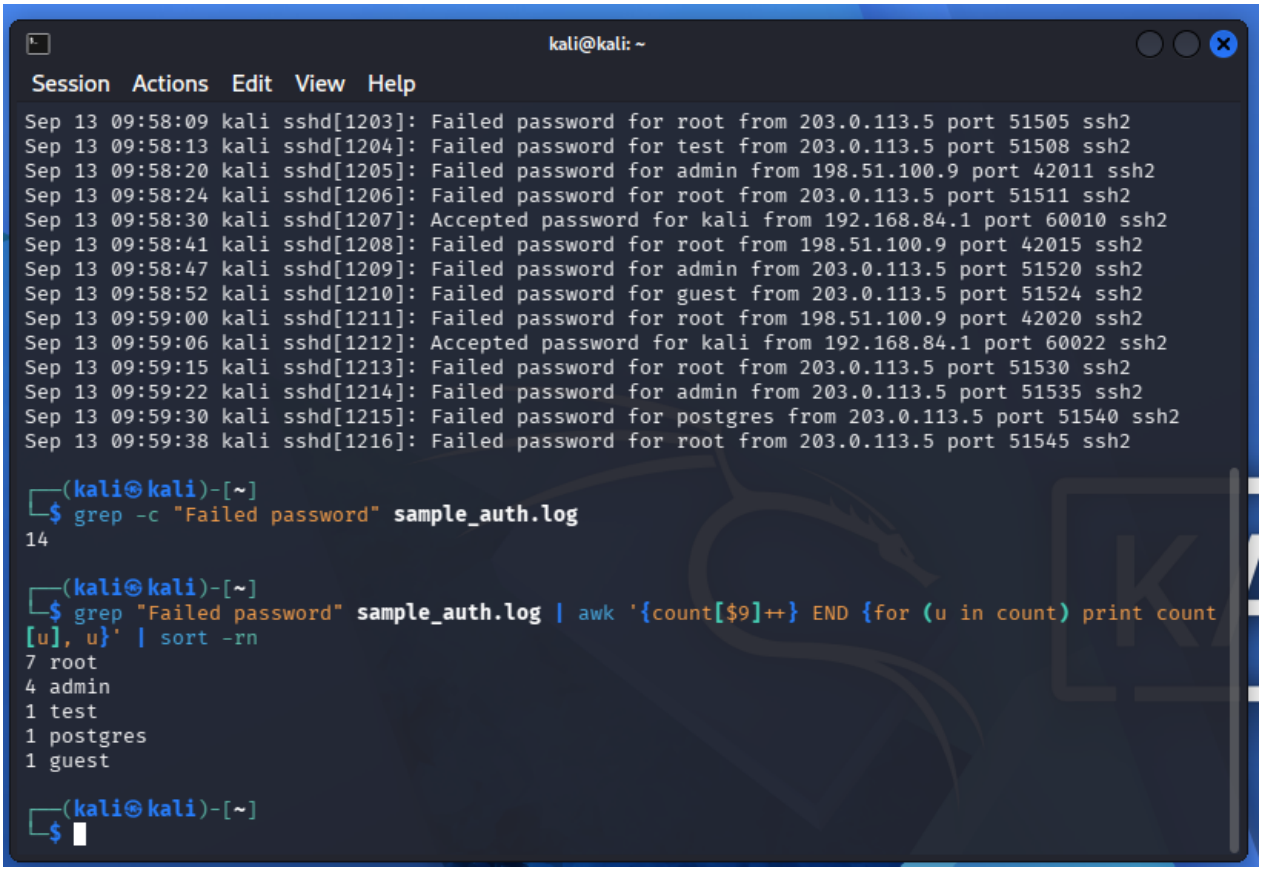
  Result: 14.



grep -c confirming 14 lines contain the failure message Failed password.

- **Step 4 - counted repeated usernames:** ran `grep "Failed password" sample_auth.log | awk '{count[$9]++} END {for (u in count) print count[u], u}' | sort -rn`.

  Result: root 7, admin 4, test 1, postgres 1, guest 1.

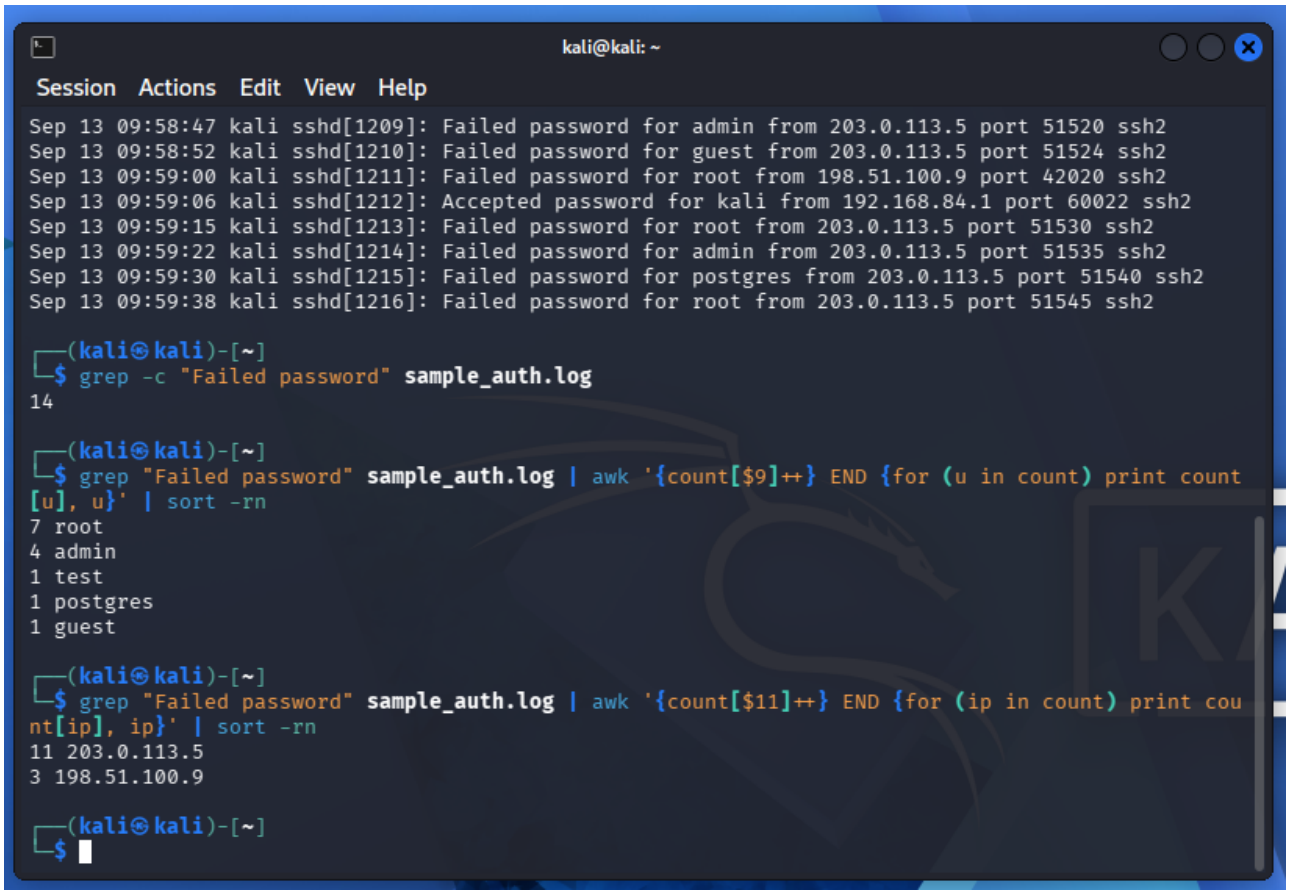


Usernames ranked by frequency: root with 7 failures, admin with 4, and three other usernames with 1 each.

root was targeted almost twice as often as admin, and every other username, test, postgres, and guest, appears only once each. That shape, one heavily repeated target plus several one-off attempts, is a recognizable dictionary-attack pattern, systematically working through a list of likely usernames, rather than a single person mistyping their own credentials.

- **Step 5 - counted repeated source IPs:** ran the same pattern against field 11, `grep "Failed password" sample_auth.log | awk '{count[$11]++} END {for (ip in count) print count[ip], ip}' | sort -rn`.

  Result: 203.0.113.5 with 11 failures, 198.51.100.9 with 3.

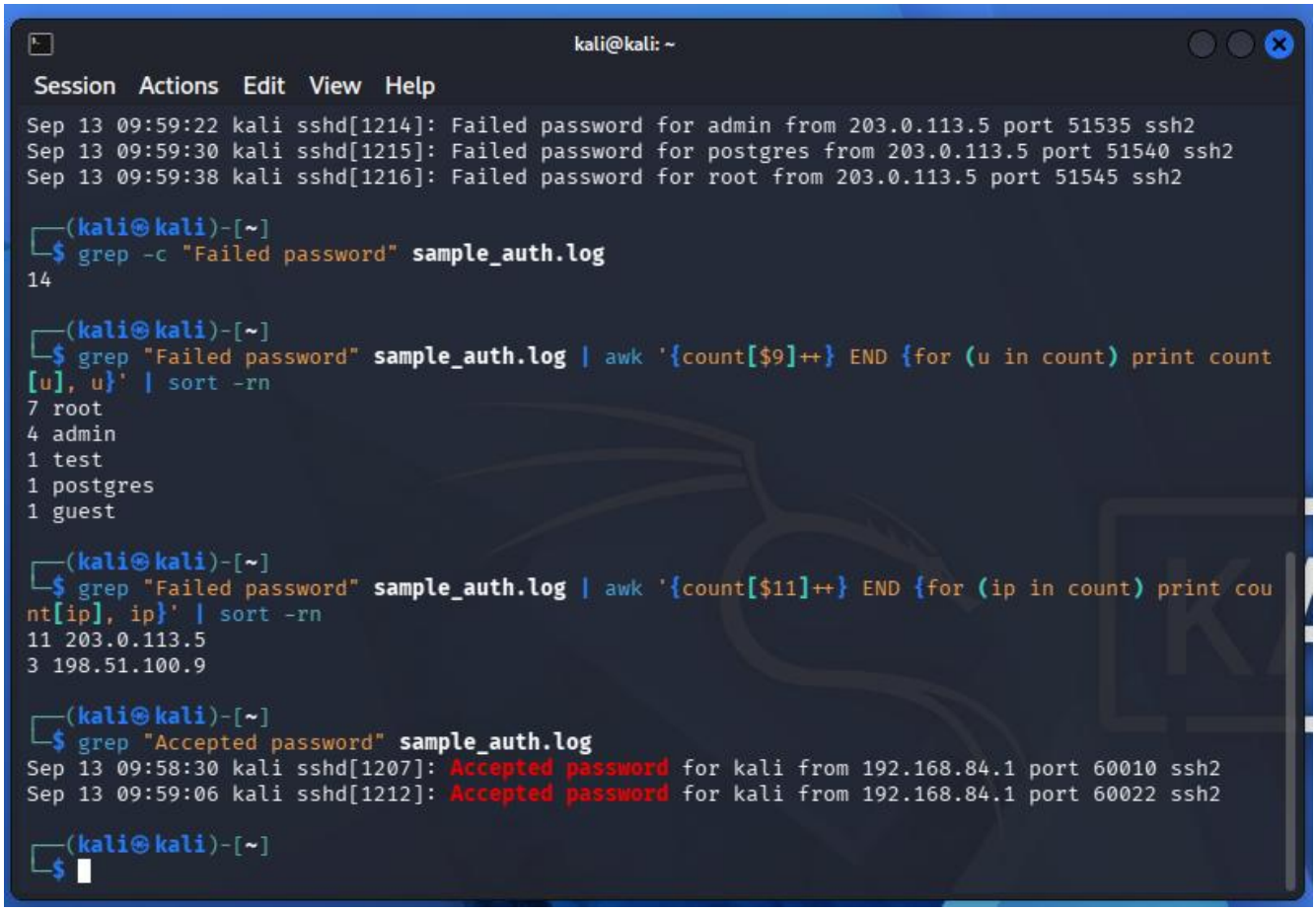


Source IPs ranked by frequency: 203.0.113.5 with 11 failures, 198.51.100.9 with 3.

203.0.113.5 accounts for eleven of the fourteen total failures, nearly four times as many as the second address. This is the strongest single finding in the dataset: one source is responsible for the overwhelming majority of failed attempts, not a handful of unrelated machines each failing occasionally.

- **Step 6 - reviewed the successful logins for contrast:** ran `grep "Accepted password" sample_auth.log`.

  Result: two successful logins, both for the account kali, both from 192.168.84.1.



Both successful logins traced to the same legitimate account and address: kali from 192.168.84.1.

Both successful logins belong to the same account, kali, from the same address, 192.168.84.1, an internal address distinct from both attacking IPs. This contrast matters: the only account that ever succeeded never appears anywhere in the failure counts at all, which supports reading the failures as an external attempt to guess in, rather than a legitimate user struggling with their own password.

8. Pattern Summary

Pattern Type	Top Result	Count	Interpretation
Repeated failure message	"Failed password"	14 of 16 lines	Overwhelming majority of the log is failed authentication, not routine activity
Repeated username	root	7 (admin next at 4)	Dictionary-style targeting of high-value account names
Repeated source IP	203.0.113.5	11 (second IP at 3)	Single source responsible for most failures; likely primary attacker
Successful login	kali from 192.168.84.1	2	Legitimate account and address, does not overlap with either failing pattern

9. Findings

9.1 One dominant IP correlates with the two most-targeted usernames

203.0.113.5 was not simply the noisiest address, cross-referencing the two result sets shows it produced 5 of the 7 root attempts and 3 of the 4 admin attempts, the two most repeated usernames in the whole log. A repeated IP and a repeated username are each a moderate signal alone; the same IP driving the two most repeated usernames together is considerably stronger evidence of a single, deliberate actor than either finding would be in isolation.

9.2 The second source was smaller in volume, not narrower in targeting

198.51.100.9 produced 3 failures: two against root and one against admin, the same two usernames the primary IP favored, just far fewer attempts overall. This means the second IP was not restricted to a single username, it shows the same general targeting pattern as the first, only at roughly a quarter of the volume. Whether this represents a second, smaller actor or the same actor briefly using a different network path is not something this log can determine on its own, and stating that honestly is preferable to guessing a specific cause the data cannot support.

9.3 The successful account never appears in either failure ranking

If the kali account had also appeared among the failed attempts, that would suggest the attacker was specifically guessing at a known valid username. Since kali never appears as a failure target at all, and its successful logins came from a distinct internal address, the evidence points toward two unrelated populations in this log, an external party guessing at generic or high-value usernames, and a legitimate user who never had any authentication trouble at all.

9.4 awk's associative arrays performed the counting without needing a separate tool

The task's tool list names grep, awk, and sort, not uniq, which is the utility most commonly paired with sort for this kind of counting in everyday use. Building the counts directly inside awk using count[$9]++ and an END block produced the same result while staying strictly within the specified toolset, and demonstrates that awk is capable of tallying and aggregating data on its own, not only extracting or reformatting fields.

10. Conclusion

A small, deliberately constructed and openly disclosed synthetic authentication log produced a clear, correlated finding using only grep, awk, and sort: one source IP, responsible for the large majority of failures, concentrated against the same two usernames and a smaller second source, also targeted, entirely separate from the one account that ever logged in successfully.

The value of this task was in building the detection logic itself, field-based counting with awk's associative arrays, message filtering with grep, and frequency ranking with sort, against data where the intended answer was already known, which is the same validation step a real detection rule would need before ever being pointed at genuine production logs.